

✓ BPL_TEST2_Batch script with OMSimulator

The key library OMSimulator is installed.

After the installation a small application BPL_TEST2_Batch is loaded and run. You can continue with this example if you like.

```
!lsb_release -a # Actual VM Ubuntu version used by Google
```

```
No LSB modules are available.
Distributor ID: Ubuntu
Description:   Ubuntu 24.04.5 LTS
Release:      24.04
Codename:     noble
```

```
!python --version
```

```
Python 3.13.15
```

```
!pip install OMSimulator
```

```
Requirement already satisfied: OMSimulator in /usr/local/lib/python3.13/dist-packages (3.0.0.post116)
Requirement already satisfied: requests in /usr/local/lib/python3.13/dist-packages (from OMSimulator) (2.32.4)
Requirement already satisfied: lxml>=5.3.1 in /usr/local/lib/python3.13/dist-packages (from OMSimulator) (6.1.2)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests->OMSi
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests->OMSi
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests->OMSi
```

```
!pip install DyMat
```

```
Requirement already satisfied: DyMat in /usr/local/lib/python3.13/dist-packages (0.7)
```

✓ BPL_TEST2_Batch setup

Now specific installation and the run simulations. Start with connecting to Github. Then upload the two files:

- FMU - BPL_TEST2_Batch_linux_om_me.fmu
- Setup-file - BPL_TEST2_Batch_fmпы_explore.py

```
%bash
git clone https://github.com/janpeter19/BPL_TEST2_Batch
```

```
fatal: destination path 'BPL_TEST2_Batch' already exists and is not an empty directory.
```

```
CalledProcessError                                Traceback (most recent call last)
/tmp/ipykernel_6897/587047059.py in <cell line: 0>()
----> 1 get_ipython().run_cell_magic('bash', '', 'git clone https://github.com/janpeter19/BPL_TEST2_Batch\n')
```

```
----- 4 frames -----
<decorator-gen-103> in shebang(self, line, cell)
/usr/local/lib/python3.13/dist-packages/IPython/core/magics/script.py in shebang(self, line, cell)
    243     sys.stderr.flush()
    244     if args.raise_error and p.returncode!=0:
--> 245         raise CalledProcessError(p.returncode, cell, output=out, stderr=err)
    246
    247     def _run_script(self, p, cell, to_close):
```

```
CalledProcessError: Command 'b'git clone https://github.com/janpeter19/BPL_TEST2_Batch\n'' returned non-zero
exit status 128.
```

Next steps: [Explain error](#)

```
%cd BPL_TEST2_Batch
```

```
/content/BPL_TEST2_Batch
```

✓ BPL_TEST2_Batch - demo

```
import OMSimulator as oms
```

```
import scipy.io
```

```
import DyMat
```

```
import matplotlib.pyplot as plt
```

Basic simulation

```
fmu = oms.FMU('BPL_TEST2_Batch_linux_om_me.fmu')
```

```
fmu.modelName
```

```
'BPL.Examples_TEST2.Batch'
```

```
for s in fmu.states:  
    print(s.name)
```

```
bioreactor.V  
bioreactor.m[1]  
bioreactor.m[2]
```

```
fmu.instantiate()
```

```
fmu.setResultFile('result.mat')
```

```
fmu.setValue('bioreactor.V_start', 1.0)  
fmu.setValue('bioreactor.m_start[1]', 1.0)  
fmu.setValue('bioreactor.m_start[2]', 10.0)
```

```
fmu.initialize()
```

```
fmu.simulate()
```

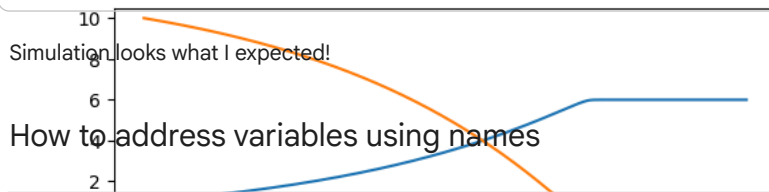
```
mat_data = scipy.io.loadmat('result.mat')
```

```
mat_data.keys()
```

```
dict_keys(['Aclass', 'name', 'description', 'dataInfo', 'data_1', 'data_2'])
```

```
t = mat_data['data_2'][0];
```

```
plt.figure()  
  
ax1 = plt.subplot(2,1,1)  
ax2 = plt.subplot(2,1,2)  
  
ax1.plot(t, mat_data['data_2'][2])  
ax1.plot(t, mat_data['data_2'][3])  
ax2.plot(t, mat_data['data_2'][12])  
plt.show()
```



How to address variables using names

```
res = DyMat.DyMatFile('result.mat')
```

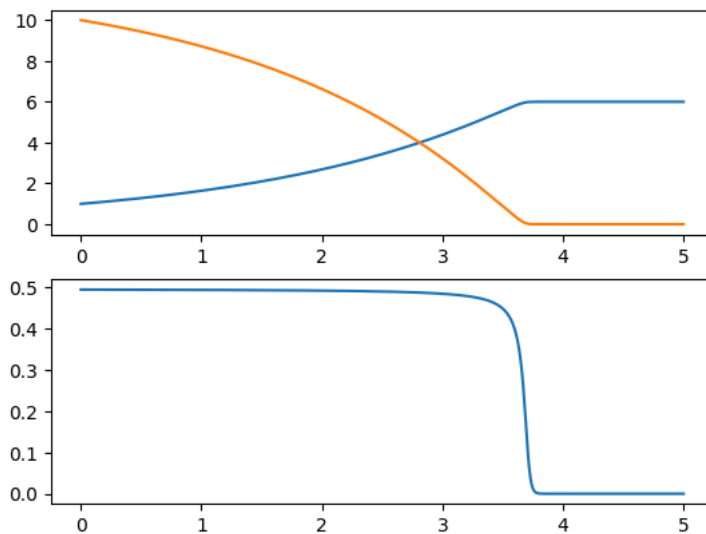
```
print(res.names())
```

```
dict_keys(['Batch.bioreactor.V', 'Batch.bioreactor.m[1]', 'Batch.bioreactor.m[2]', 'Batch.der(bioreactor.V)', 'Ba
```

```
plt.figure()
```

```
ax1 = plt.subplot(2,1,1)
ax2 = plt.subplot(2,1,2)
```

```
ax1.plot(t, res['Batch.bioreactor.m[1]'])
ax1.plot(t, res['Batch.bioreactor.m[2]'])
ax2.plot(t, res['Batch.bioreactor.culture.mu'])
plt.show()
```



How to address "time" in the res-file?

Start coding or [generate](#) with AI.